Universidad Laica “Eloy Alfaro de Manabí”

Materia:

Aplicaciones Móviles Nativas

Docente:

Ing. Edgardo Panchana

Estudiantes:

Jhonny Castillo Mero

Carrera:

Ingeniería de Software

Curso:

6to Nivel "A"

Año:

2026

INFORME DE PRÁCTICA EXPERIMENTAL DE APRENDIZAJE (PEA)

ViewModel y Ciclo de Vida en Dessert Clicker

(Resolución integrada de Codelabs: Activity Lifecycle & Practice: Add a ViewModel)

🔗 **REPOSITORIO GIT PÚBLICO DEL PROYECTO:**

https://github.com/JhonnyCCM/DessertClicker

Clonación: `git clone https://github.com/JhonnyCCM/DessertClicker.git`

1. Descripción General y Alcance de las Actividades

El presente laboratorio consolida el desarrollo de dos actividades pedagógicas consecutivas de Google Android Developers, cuyo código fuente completo, estructura y configuración de dependencias se encuentran alojados en el repositorio público de GitHub.

• Actividad 1 - Stages of the Activity Lifecycle: Se analiza el comportamiento de la Activity durante la interacción del usuario y se implementa el registro de logs en Logcat con Log.d() para cada uno de los métodos de ciclo de vida (onCreate, onStart, onResume, onPause, onStop, onDestroy, onRestart). Se evidencia el problema de destrucción y pérdida de estado ante la rotación de pantalla.

• Actividad 2 - Practice: Add a ViewModel to Dessert Clicker: Se refactoriza la arquitectura de la aplicación desacoplando el estado y las reglas de negocio de MainActivity hacia una clase DessertViewModel respaldada por StateFlow y el modelo de datos inmutable DessertUiState.

Con esto se logra que los datos persistan de forma natural e inmune a los cambios de configuración sin necesidad de sobrecargar la UI.

2. Actividad 1: Ciclo de Vida de la Actividad (Logcat y Callbacks)

En esta primera parte, se instrumentaron los métodos de callback del ciclo de vida en MainActivity.kt. Durante la ejecución en el emulador Google Pixel 10 Pro XL (API 35), Logcat registró fielmente cada transición de estado:

Registro en Logcat (Filtro tag:MainActivity):

```
10-14 15:21:16.391  D/MainActivity: onCreate Called  --> Inicialización e inflado de UI
10-14 15:21:16.463  D/MainActivity: onStart Called   --> App visible en pantalla
10-14 15:21:16.503  D/MainActivity: onResume Called  --> Foco e interacción táctil habilitada
--- Cambio de Configuración: Rotación de Pantalla (Portrait a Landscape) ---
10-14 15:22:03.011  D/MainActivity: onPause Called   --> Pérdida de foco
10-14 15:22:03.032  D/MainActivity: onStop Called    --> Actividad oculta
10-14 15:22:03.038  D/MainActivity: onDestroy Called --> Destrucción total de la Activity previa
--- Recreación inmediata por el Sistema Operativo ---
10-14 15:22:03.137  D/MainActivity: onCreate Called  --> Nueva instancia creada
10-14 15:22:03.139  D/MainActivity: onStart Called   --> Visible nuevamente
10-14 15:22:03.139  D/MainActivity: onResume Called  --> Foco restablecido
```

Explicación técnica de la Actividad 1:

Cuando el usuario rota el dispositivo, Android destruye completamente la Activity (onDestroy) y la vuelve a crear (onCreate) para cargar los recursos que correspondan a la nueva orientación. En una implementación tradicional con variables locales, este ciclo resetea la recaudación y las ventas a 0. Esto demuestra la necesidad indispensable de la Actividad 2.

3. Actividad 2: Implementación de ViewModel y Evidencias Visuales

En la segunda actividad se resolvió la pérdida de datos mediante la arquitectura MVVM recomendada por Android. Toda la lógica de recaudación, conteo de ventas y selección del siguiente postre se trasladó a DessertViewModel, mientras que la vista en Jetpack Compose observa el estado reactivo mediante StateFlow y collectAsState().

A continuación, se presentan las capturas de pantalla obtenidas en el emulador Google Pixel 10 Pro XL, acompañadas de la explicación puntual de qué se está realizando, por qué ocurre y en qué fase de la actividad nos encontramos:

Captura 1: Estado Inicial del Juego (Cupcake)



• ¿En qué parte de la actividad estamos?

Inicio de la Actividad 2: Primera carga de la UI tras la inicialización del ViewModel.

• ¿Qué se está haciendo?

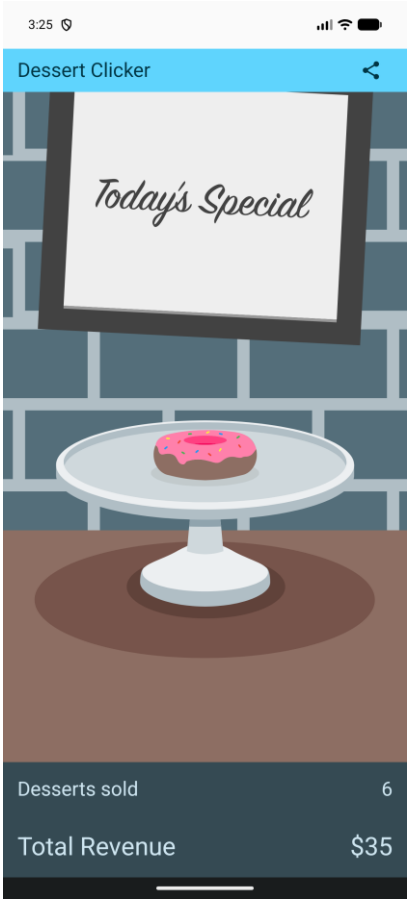
La app presenta el estado base inicial: imagen del Cupcake, Total Revenue en $0 y Desserts Sold en 0.

• ¿Por qué ocurre?

DessertViewModel inicializa _dessertUiState con los valores por defecto de la data class

DessertUiState() (índice 0, precio $5). La vista Compose observa este StateFlow inicial a través de collectAsState() y renderiza la pantalla sin necesidad de lógica inline.

Captura 2: Primera Transición Dinámica de Postre (Donut)



- **¿En qué parte de la actividad estamos?**

Fase de interacción táctil y procesamiento de eventos en el ViewModel.

- **¿Qué se está haciendo?**

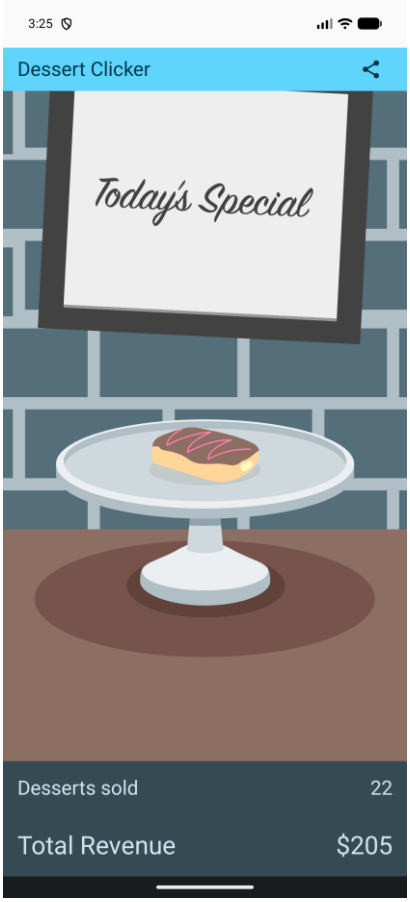
Tras pulsar el postre 6 veces, la imagen cambia automáticamente a la Dona (Donut), las ventas suben a 6 y el ingreso acumulado alcanza $35.

- **¿Por qué ocurre?**

Cada clic despacha el evento viewModel.onDessertClicked(). El ViewModel suma el precio actual ($5 por cupcake), incrementa dessertsSold y ejecuta

determineDessertIndex(). Como dessertsSold >= 5 (umbral de producción de la dona), el ViewModel emite un nuevo DessertUiState con la imagen de la dona y nuevo precio ($10). Compose recibe la emisión y recompone la pantalla reactivamente.

Captura 3: Escalado de Producción a Nivel Superior (Éclair)



- **¿En qué parte de la actividad estamos?**

Validación del algoritmo de negocio en DessertViewModel.

- **¿Qué se está haciendo?**

Con 22 postres vendidos y una recaudación de $215, la interfaz muestra el tercer postre del catálogo: el Éclair.

- **¿Por qué ocurre?**

Demuestra la correcta evaluación de la lista Datasource.dessertList dentro de determineDessertIndex(). Al superar las 20

ventas, el algoritmo selecciona el índice 2 (Éclair, precio $15). El estado inmutable se mantiene íntegro y sin desbordamientos ni inconsistencias de cálculo.

Captura 4: Prueba Crítica de Cambio de Configuración (Rotación a Landscape)



• ¿En qué parte de la actividad estamos?

Comprobación final y validación de la arquitectura MVVM frente a la problemática de la Actividad 1.

• ¿Qué se está haciendo?

Se rota el dispositivo físico o emulador a modo horizontal. La interfaz se readapta fluidamente manteniendo exactamente los $215 de ingreso y los 22 postres vendidos.

• ¿Por qué ocurre?

A diferencia de la Actividad 1 donde la destrucción de la Activity destruía los datos, el ViewModel vive en el ViewModelStore.

Cuando el sistema operativo destruye la MainActivity anterior y genera la nueva en orientación landscape, el método viewModel() reconecta la nueva composición a la misma instancia viva del ViewModel en memoria, preservando el 100% del estado sin reiniciar valores.

4. Conclusiones

1. Desacoplamiento Exitoso: Se logró una arquitectura limpia donde MainActivity actúa exclusivamente como vista declarativa, delegando el cálculo y las decisiones de negocio al ViewModel.

2. Resiliencia ante Cambios de Configuración: La rotación demostró en la práctica el propósito fundamental de los ViewModels en Android: garantizar que el estado sobreviva a recreaciones del ciclo de vida sin depender de almacenamiento manual engorroso.

3. Trazabilidad Técnica: El repositorio público en GitHub contiene la solución compilable y verificada bajo los estándares de Material 3 y Jetpack Compose.